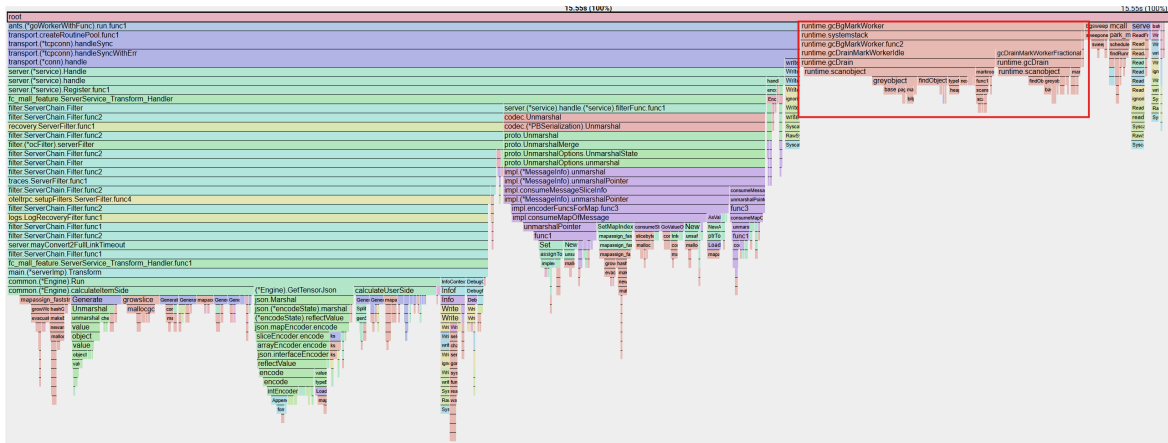


20250806日报

今日工作内容

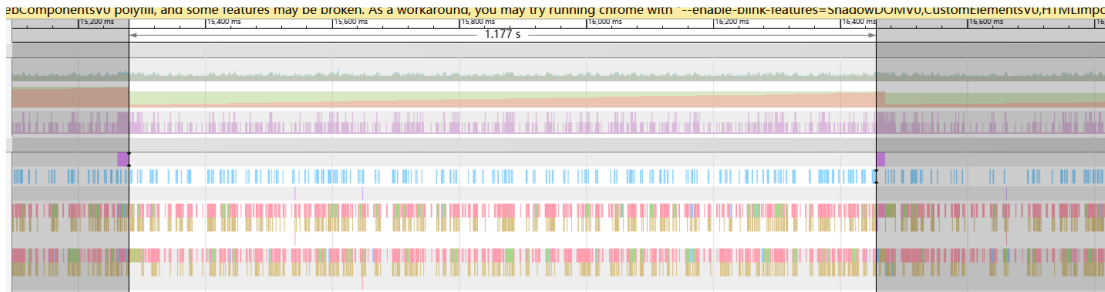
1. 深度分析之前的fc_mall_feature服务中runtime.gcBgMarkWorker部分；



runtime.gcBgMarkWorker占比较大，直接反应的是Go程序的垃圾回收（GC）标记阶段开销过大。runtime.gcBgMarkWorker 的CPU占比过高，既可能因临时对象过多导致单次GC标记工作量大，也可能因GC触发频率过高导致频繁扫描。

分析与验证过程：

1、view trace by proc（分析GC频率）：



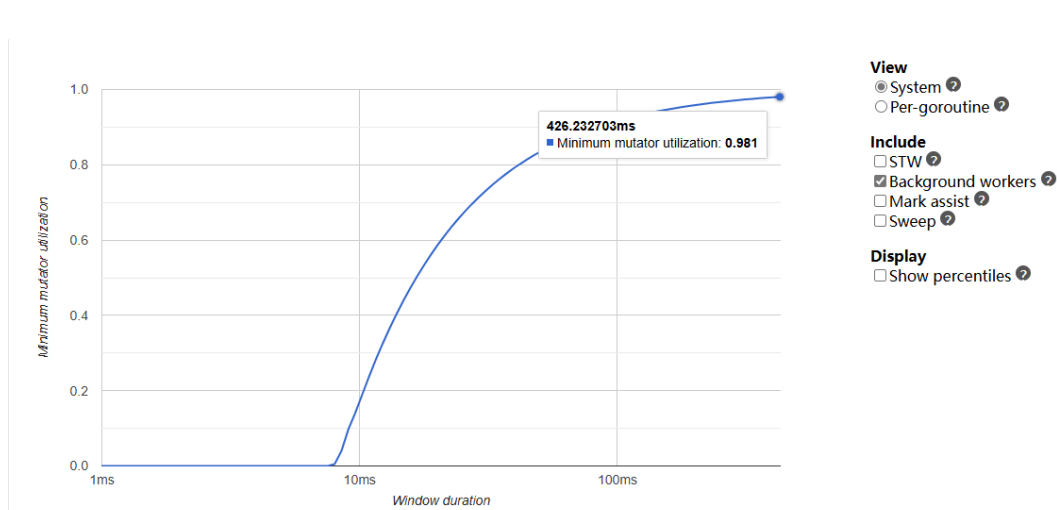
上述的trace图中Heap堆内存呈现锯齿状，GC会在每次下降沿出现，很明显，GC在不停的回收内存。图中显示了GC之间的间隔为1.17s左右，即每隔1.17s才会进行回收内存。

Counter	Series	Time	Value
Heap	NextGC	16476.915584000002	238452464
Heap	Allocated	16476.915584000002	25706336

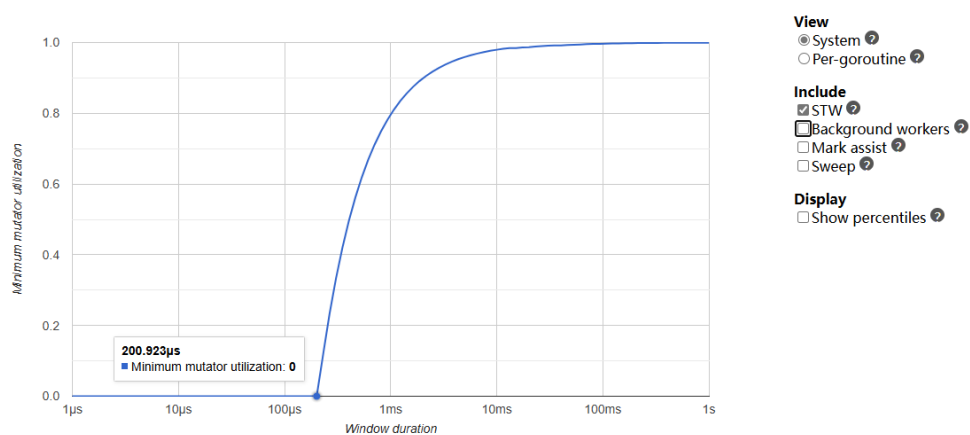
找到Trace图中最低点，从内存角度理解，可以发现，每轮GC大约有230MB内存可以用，用完了就触发GC。

2、Minimum mutator utilization：

a) Background workers



b) STW



x轴表示取的时间窗口，y轴表示这个时间窗口里，最差情况下有多少比例的CPU时间可供业务代码使用。从结果中看出STW几乎不占用cpu资源，而Background workers在400ms中占用1.9%左右，而在1s中就大概有20ms。

3、内存分配占比：

```

sh-4.4# go tool pprof -seconds 30 -alloc objects 30.175.207.88:8091/debug/pprof/allocs
Fetching profile over HTTP from http://30.175.207.88:8091/debug/pprof/allocs?seconds=30
Please wait... (30s)
^ESaved profile in /root/pprof/pprof.fc_mall_feature.alloc_objects.alloc_space.inuse_objects.inuse_space.005.pb.gz
File: fc_mall_feature
Type: alloc objects
Time: Aug 6, 2025 at 5:33pm (CST)
Duration: 30.04s, Total samples = 59790755
Entering interactive mode (type "help" for commands, "o" for options)
(pprof) o
call_tree           = false
compact_labels      = true
cumulative           = flat           //: [cum | flat]
divide_by           = 1
drop_negative       = false
edgefraction        = 0.001
focus              = ""
granularity         = filefunctions   //: [addresses | filefunctions | files | functions | lines]
hide                = ""
ignore              = ""
mean                = false
nodecount           = -1              //: default
nodefraction        = 0.005
noinlines           = false
normalize           = false
output              = ""
prune_from          = ""
relative_percentages = false
sample_index        = alloc_objects   //: [alloc_objects | alloc_space | inuse_objects | inuse_space]
show                = ""
show_from           = ""
tagfocus            = ""
taghide             = ""
tagignore           = ""
tagshow             = ""
trim                = true
trim_path           = ""
unit                = minimum
(pprof) sample_index=alloc_space
(pprof) top 10
Showing nodes accounting for 3500.72MB, 64.40% of 5435.54MB total
Dropped 219 nodes (cum <= 27.18MB)
Showing top 10 nodes out of 93

```

flat	flat%	sum%	flat	flat%	sum%	function
1116.24MB	20.54%	20.54%	1523.76MB	28.03%		fc_mall_feature/generators/common.(*Engine).calculateItemSide
625.98MB	11.52%	32.05%	625.98MB	11.52%		reflect.mapassign_faststr0
396.52MB	7.29%	39.35%	396.52MB	7.29%		reflect.New
276.25MB	5.08%	44.43%	276.25MB	5.08%		git.code.oa.com/trpc-go/trpc-go.(*framer).ReadFrame
269.11MB	4.95%	49.38%	269.11MB	4.95%		go.uber.org/zap/buffer.(*Buffer).AppendString
223.17MB	4.11%	53.49%	827.59MB	15.23%		fc_mall_feature/generators/common.(*Engine).calculateUserSide
159.56MB	2.94%	56.42%	159.56MB	2.94%		strings.genSplit
154MB	2.83%	59.26%	154MB	2.83%		fc_mall_feature/generators/common.NewKvList
147.29MB	2.71%	61.96%	147.29MB	2.71%		fmt.Sprintf
132.60MB	2.44%	64.40%	132.60MB	2.44%		fc_mall_feature/generators/fcmallhot/kernels.Clip[go.shape.int64] (inline)

```

(pprof)

```

从结果可知，calculateItemSide()函数自身的代码（flat%）占用了20.54%的内存，即在函数内部有大量的临时变量分配。因此，可以考虑优化此函数内部的变量，使它们尽量复用，从而达到降低runtime.gcBgMarkWorker的目的。

需要注意的是runtime.gcBgMarkWorker的优化没有真正实现，只是在这做了一个分析和验证，如果后续需要在此点上进行优化，可以基于此文档进行后续开发。

明日待办：

- 1、基于佐贤哥发的gdp接口，继续开发“线上go+python服务正式环境不允许debug日志级别”需求；